2025 8th International Conference on Advanced Electronic Materials, Computers and Software Engineering (AEMCSE) | 979-8-3315-1091-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/AEMCSE65292.2025.11042761

Intent-driven Web UI Tests Repair with LLM

Yingjie Tao
College of Information Science and Technology
Beijing University of Chemical Technology
Beijing, China
2022200842@buct.edu.cn

Weiwei Wang
College of Information Engineering
Beijing Institute of Petrochemical Technology
Beijing, China
wangweiwei@bipt.edu.cn

Junxia Guo*
College of Information Science and Technology
Beijing University of Chemical Technology
Beijing, China
*gjxia@mail.buct.edu.cn

***Abstract*—As web applications are frequently updated, changes may introduce to web elements in new versions, causing test cases to fail. Consequently, automatic web test repair techniques are proposed to reduce the cost of regression testing. Most existing methods focus on finding the correct candidate elements or related attributes to fix the broken test case. However, when test case failures are caused by test flow changes or propagated breakages, those methods that focus solely on matching the failing element in the new version cannot work well. Through empirical analysis, we found that the test intent and the reasons that caused the test failure are useful in test case reparation. This paper proposes a novel intent-driven web test repair approach named LetTe, which first parses the test intent and failure reasons of failed web UI tests, and then guides the Large Language Model (LLM) to fix them via prompt design and fine-tuning. LetTe's repair logic is to simulate that of a human expert. According to the test intent and reason for failure, "think about" the possible repair plan and then generate repair candidates through the corresponding chain-of-thought. We evaluate LetTe on 7 web applications collected from open-source websites as well as publicly available datasets. The experimental results show that our approach has a 75% correct repair rate, which is higher than all baseline methods.**

Keywords-Web Application Testing, Test Intent, Test Case Repair, Large Language Model

I. Introduction

Today's web applications are feature-rich and user-friendly, meanwhile constantly evolving and updating. With the version iteration, testing of web applications needs to go along. In the process of updating, some changes may cause the test cases of the old version to be invalidated. Directly discarding the broken test cases leads to degrading the quality of the test suite [1]. According to statistics from Pinto et al. [2], 29% of test cases involving changes in software evolution were obtained by fixing test cases. But the cost of manually repairing test cases is high. Thus, how to implement automatically web test repair is a research hotspot.

Research [3] shows that 73.62% of test failures are related to locators in web application testing. So, fixing the locators becomes the most direct solution. This kind of solutions [4-7] usually repair test cases through matching elements and related information, neglecting the causes analysis of test failures and whether the repaired test case can fulfill the test intent. This leads to a large number of ineffective repairs. For example, a test case *AddUserTest* in the experiments of Lin et al. [5] is to test a web application named *MantisBT*, which is a popular open-source bug tracker system. In the old version of *MantisBT*, the workflow of adding a new user includes two steps: 1) clicking the "Manage" button, and the page transfers to the "Manage Users" page; 2) clicking the button of "create new account". But in the new version, this workflow changes, and it includes three steps: 1) clicking the "Manage" button in the navigation bar; 2) clicking the "Manage Users" button; 3) finding the "Create New Account" button and clicking it. Due to this change, the test case fails in the new version. According to their experimental results, the existing test repair methods like WATER [4], VISTA [6], WEBEVO [8], SFTM [9], and their method UITESTFIX [5] are all unable to fix this test case. The key reason is that the "Create New Account" button or any similar elements do not exist on the page navigated to after clicking on the "Manage" button in the new version, but it does in the old version. In this situation, without appropriate guidance, the existing repair methods that focus on local failure points are powerless. But, if we consider the global intent of the test case, it will add *clicking the "Manage Users" button* to the workflow. In addition, if the failure causes are fully explored, it can also help identify the missing step. Thus, this paper integrates test intent and feedback information on failure reasons to guide the repair of test cases.

LLM's excellent performance in semantic analysis and reasoning capabilities, as well as its rich prior knowledge of related domains, make it well suited for analyzing the test intent and failure causes of web test cases. However, without professional and effective customized prompts, it is difficult for LLM to capture test flow changes or propagation breakages, let alone provide test fixes in that situation.

Therefore, this paper proposes a test intent-driven Web UI test repair technique called LetTe, which guides repair through test intent and broken test analysis. Concretely, as the way a human expert would think when encountering a broken test case, we first analyze the possible causes of failure by leveraging the changes brought by the updated version. Then analyze the corresponding test intent for the original test case by extracting and summarizing the test behavior on the old version of the web application, which contains the functionality being tested and test steps. Next, guided by the test intent and the failure reason, we leverage chain-of-thought reasoning to fix the broken test. The contributions of this paper are as follows:

- Propose a test intent-driven approach LetTe, which first analyzes the test intent and failure reasons of test cases and then uses them to guide the generation of repair candidates.
- Construct a fine-tuning model for fixing broken test cases oriented web applications, based on the chain-of-thought reasoning of manual repair logic.
- LetTe achieves 85.05% and 75% correct repair rate in the two datasets, respectively. The number of correctly repaired test cases is the highest among existing methods.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:51:56 UTC from IEEE Xplore. Restrictions apply.

II. LETTE APPROACH

We propose a test intent-driven web application test case repair approach LetTe. Figure 1 briefly describes our approach, which consists of two phases: (I) *Broken Test Analysis*, which analyzes the test intent of the original test case and the causes of failure in the new version by comparing the major elements of the page in the old and new versions, serving as a guidance for the repair of the next stage; (II) *Broken Test Repair*, which generates repair candidates of broken test cases via the chain-of-thought reasoning containing the repair logic guided by the test intent and failure reasons. In order to take into account special cases such as the unfixable situation, LetTe first leverages the fine-tuned model to generate repair candidates, and if it fails to generate the correct repair candidate, it further determines whether the test case intent can be reproduced. When the result of the judgment is that it may still be repairable, then it will try again. Otherwise, we think that the page changed significantly when updating the web application that it is beyond repair.

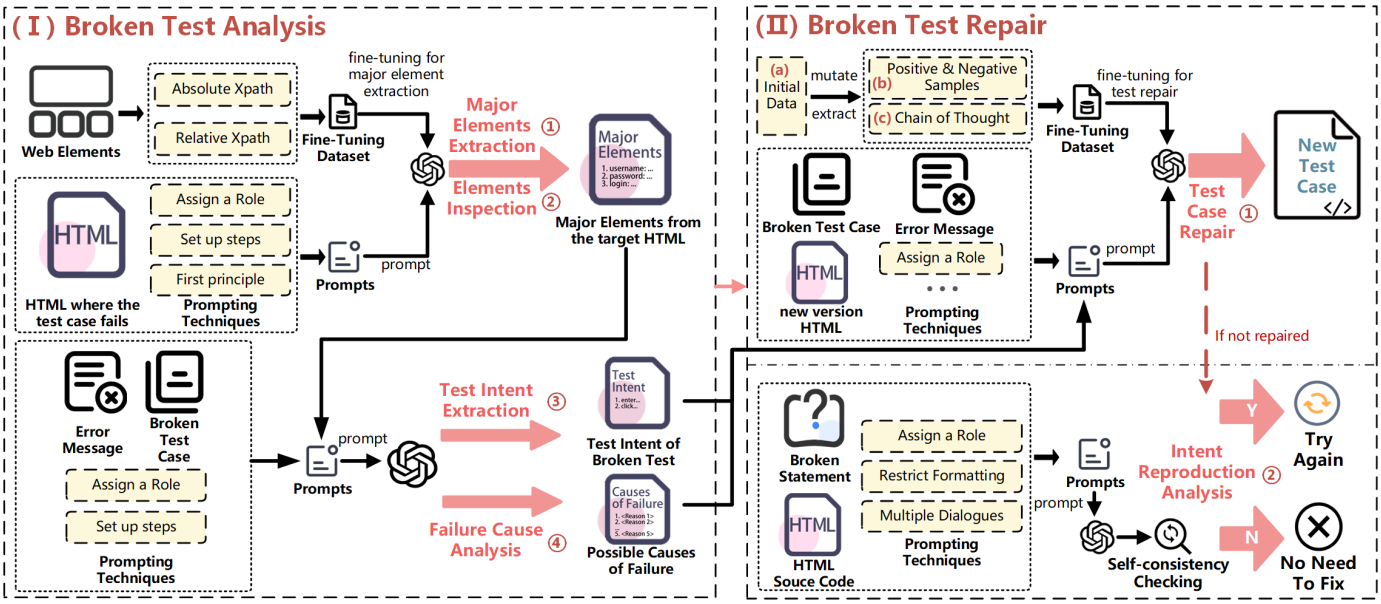

Figure. 1. Framework of LetTe.

Since LetTe leverages the capabilities of LLM, we use the official documents like ``OpenAI's Six strategies for getting better results'' as references when designing prompts. In Table 1, we summarized the prompting techniques we used in our approach and numbered them with RIDs to make it easier to explain the techniques for each sub-task later on.

A. *Broken Test Analysis*

Considering that before repairing the test case manually, human experts would analyze the test intent and failure causes by inspecting the test case name, observing the test process of the original test case, and comparing the differences between the old and new versions of the page. Thus, the *Broken Test Analysis* phase of LetTe consists of four modules, ① Major Elements Extraction, ② Elements Inspection, ③ Test Intent Analysis, ④ Failure Cause Analysis. These modules are designed to simulate the analysis process carried out by human experts before fixing the test cases. Our example conversation log can be seen in Figure 2. From this figure, we can see that LetTe extracts the necessary information from the broken test case and error messages, analyzes the test intent and possible reasons causing the failure, which provides reference information for subsequent test case repair.

TABLE 1. PROMPTING TECHNIQUES FOR INSPIRING DESIGN

RID	Tactic in OpenAI's official documentation
R1	Ask the model to adopt a persona.
R2	Specify the steps required to complete a task.
R3	Instruct the model to work out its own solution before rushing to a conclusion.
R4	Use delimiters to clearly indicate distinct parts of the input.
R5	Use inner monologue or a sequence of queries to hide the model's reasoning process.

With the help of prompting techniques mentioned in the Table 1, our prompt rules for the *Broken Test Analysis* phase are as follow:

Major Elements Extraction Prompt: Techniques[R1, R2] + HTML Source Code

Elements Inspection Prompt: Techniques[R1, R3] + HTML Source Code + Major Elements and Xpath

Test Intent Analysis Prompt: Techniques[R1] + Filename + Original Test Case + Checked Major Elements and Xpath

Failure Cause Analysis Prompt: Techniques[R1, R2] + Major Elements of Old Version + Major Elements of New Version + Error Message

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:51:56 UTC from IEEE Xplore. Restrictions apply.

Figure. 2. Example of how LetTe works in the BrokenTestAnalysis phase.

1) *Major Elements Extraction*

In this module, we extract the major interactive elements (e.g. buttons, input boxes) from the simplified HTML source code, and the corresponding Xpath expressions for these elements. An example is shown in Table 2. We define "major interactable elements" as form elements or table elements located in the main area of a web application's interface, except for the navigation bar. Web application updates are more about updating elements or attributes of elements in the center of the page, such as forms, and rarely involve things like navigation bars, since test case failures related to the elements in the navigation bar are uncommon. In order to automatically extract the main elements of the page, we create prompt templates using prompting techniques and fine-tune the LLM in this module, which is based on zero-shot learning. In addition, we divide this work into two steps: (i) analyzing which are the main elements; (ii) listing the main elements and extracting the corresponding Xpath expressions.

TABLE 2. EXAMPLE OF EXTRACTING THE MAJOR ELEMENTS

TEXT TYPE	CONTENT
HTML Source Code	<html> <body id="admidio-overview"> ..... <label>Username< /label> <input id="login name"/ > < /div> ..... <label>Password< /label> <input id="usr password"/ > < /div> ..... <button id="next" type="submit">Sign in< /button> ..... < /body> < /html>
Major Elements	1. "Username" xpath="//*[@id="login_name"]" 2. "Password" xpath="//*[@id="usr_password"]" 3. "Sign in" xpath="//*[@id="next"]"

Fine-tuning: Natural language models are usually designed and trained based on generic tasks, and their responses are randomized. Therefore, it is likely to get inconsistent responses for the same task. In addition, web applications have multiple interactive interface styles, which causes the format of the responses to vary a lot. It cannot be addressed only through prompting techniques, even together with few-shot learning. Therefore, fine-tuning is applied to the extraction of the major elements and Xpaths to solve this problem. Concretely, we extracted 120 pieces of major elements as fine-tuning data from 60 web pages of six different web applications, from which we selected 10 pages per web application. As to these six applications, we use five open-source web applications collected by ourselves during the evaluation work (as shown in Table 5) and an e-commerce web application called *PrestaShop*. To be more specific, *Admidio* is an online membership management platform, which belongs to the backend management category of the application interface design, and the interface of the information display in the form of lists and cards; *Collabtive* is a project management application, and the information display is based on simple forms, without too many decorative elements, emphasizing practicality; *MantisBT* is an issue/bug tracking management platform, similar to project management applications, but unlike *Collabtive*, the page layout structure is very different, especially the layout of the navigation bar and form style; *MRBS* is a web application for meeting booking, similar to multi-functional calendar software; *PHP-Fusion* belongs to the content management or information display type of management system, which is similar to a blog website; *PrestaShop* has a modern e-commerce type of interface, while the other five apps are more like traditional management systems. With so much difference, we can improve the robustness and generalizability of our approach. Meanwhile, it should be noted that when we extracted fine-tuning data from the five web applications which are also used in the evaluation, we deliberately avoided pages that had attribute or structural changes in the new version. Considering the flexibility of XPath expressions, we use both relative and absolute Xpath expression, examples are shown in Table 3. We upload the data for fine-tuning on the official platform of OpenAI and set all the fine-tuning parameters to default.

TABLE 3. EXAMPLE OF EXTRACTED FINE-TUNING DATA

Xpath TYPE	EXPRESSION
Relative Xpath	1. "Username" xpath="//*[@id="login_name"]" 2. "Password" xpath="//*[@id="usr_password"]" 3. "Sign in" xpath="//*[@id="next"]"
Absolute Xpath	1. "Username" xpath="/html/body/div/form/div[1]/input" 2. "Password" xpath="/html/body/div/form/div[2]/input" 3. "Sign in" xpath="/html/body/div/form/input"

2) *Elements Inspection*

To make the extraction of the main elements more accurate, we inspect the responses generated by the *Major Elements*

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:51:56 UTC from IEEE Xplore. Restrictions apply.

Extraction module, which checks the accuracy of the previous answer and make corrections if needed.

Fine-tuning: Since this module needs to generate the answer first (re-extract the main element and Xpath), the fine-tuning model we use here is the same as that of the *Major Element Extraction* module.

3) *Test Intent Analysis*

Test intent is the central guiding principle in designing and executing test cases. When human experts are trying to repair a test case, they strictly follow the original intent of the test case to do so. Studies [10-11] have demonstrated that intent has a positive effect on tasks such as unit test repair and code comment generation. Furthermore, we expect the intent of repaired test cases to be consistent with that of the original test cases, by leveraging test intent, we can articulate a clear repair objective and direct the repair strategy accordingly. The test intent can be reflected through the test path and the specific steps involved.

In this module, we utilize prompt terms to analyze the test intent of a broken test case. We take the filename of the test case, the original test case, and the information about the main elements on the old version as inputs to this module. Test cases are generally named using the camel case naming convention, which briefly contains what the test case needs to test in natural language. Thus, the general test intent can be learned by the filename. Meanwhile, the original test cases and the information about the main elements in the old version of the page can be analyzed to find out the exact steps of the test. An example of test intent is shown in Figure 3, which contains the test steps and the overall test intent.

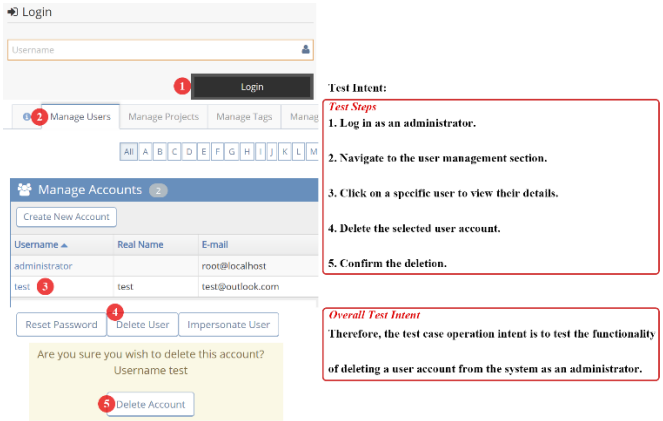

Figure. 3. The Example of a Test Intent.

4) *Failure Cause Analysis*

When a human expert is facing a test breakage, he or she first looks at and analyzes what error caused the breakage, so we set up such a module to simulate this process. In order to precisely locate the problem during test case repair, we analyze the potential causes of web test failures in this module. The input of this module are the extracted major elements of the old and new versions along with their XPaths, as well as the error messages reported when the test case breaks. Then we ask to analyze the root cause of the test case failure and other possible causes. This process involves the steps like comparing and analyzing the differences between the major elements and their XPaths in the old and new versions. Subsequently, This module will provide a specific analysis of the reasons for the test case failures.

B. *Broken Test Repair*

The second phase of LetTe is to fix the broken test case. It's important to note that when updating a web application, the developer may remove or integrate some features from the original version into other functions. As a result, test cases based on or covering this functionality are broken, and this type of broken test case cannot be repaired. For this reason, we designed two modules in the second phase, ① *Test Case Repair* module and ② *Intent Replication Analysis* module.

When LetTe encounters a broken test case, it begins with the repair process. In this case, we use the *Test Case Repair* module for the repair. When the correct candidate does not exist among the generated repair candidates, we utilize the *Intent Replication Analysis* module to analyze whether the test case has the possibility to be correctly repaired. Based on the judgment result, we provide the next step repair suggestion. If the judgment says "no need to repair", it indicates a large difference in the page, which suggests that there may be an inconsistency between the previous operation in the new version and the original test intent. This inconsistency could lead to the transition to the wrong page, making it irreparable. The example of conversation can be seen in Figure 4. And with the help of prompting techniques, the prompt rules for the *Broken Test Repair* phase are as follow:

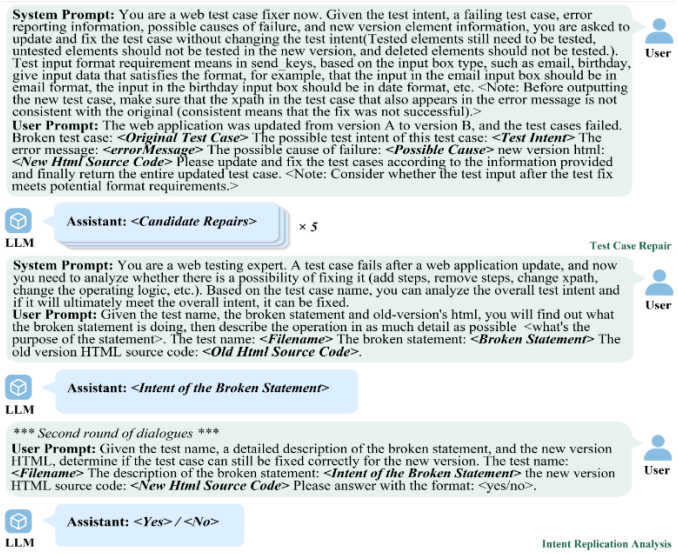

Figure. 4. Example of how LetTe works in the BrokenTestRepair phase.

Test Case Repair Prompt: Techniques[R1] + Original Test Case + Test Intent + Error Message + Possible Cause + New HTML Source Code

Intent Replication Analysis Prompt: Techniques[R1, R4, R5] + Filename + Broken Statement + Old HTML Source Code + Intent of Broken Statement + New HTML Source Code

1) *Test Case Repair*

Relying on the test intent and possible causes analyzed from the first phase, we provide repair guidance to this module. In

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:51:56 UTC from IEEE Xplore. Restrictions apply.

addition, we use the new version of the simplified HTML source code, the error message, and the broken test case as inputs to make the repairs. We integrate the idea of Tree-of-Thought (ToT) reasoning [12] in this work, which allows LetTe to generate different repair candidates for each different possible cause of failure. To be more specific, we generate five possible causes of failure for the broken test case, and for each possible cause, we perform fixing and output five candidate repairs. LetTe guides the repair process based on the test intent and cause of failure, and utilizes the page information in the new release to repair the broken test case.

Fine-tuning: Existing research work [3] has revealed that there can be 14 different reasons for test case failure in web applications, and due to the maximum token constraint of LLM, the sample of cases we can provide is very limited and it is impossible to cover all types. So the task of web test case repair is not a good case for applying the prompting method of few-shot learning. Therefore, we fine-tune the LLM used in this module. Inspired by the positive and negative samples in the deep learning training data [13], we modify the fine-tuning data and design the chain-of-thought reasoning for each sample of test case repair. As we show in Figure 1(a)~(c), the specific fine-tuning data construction steps are as follows:

(a) Construct initial fine-tuning data. We created 30 initial fine-tuning data based on six web applications, which are the same as the applications used in the *Major Element Extraction* module. For each web application, we extracted the same pages from the old and new versions, based on which we manually construct five broken test cases and the HMTL source code from the old and new versions, and we utilized the *Test Intent Analysis* module and *Failure Cause Analysis* module to obtain the corresponding test intents and the possible reasons for the failures.

(b) Construct positive and negative sample fine-tuning data. We manually checked and transformed these 30 fine-tuning data by mutating the failure causes with positive and negative samples (i.e., if the original failure cause is checked to be the correct cause analysis, a variation with a similar style but incorrect cause analysis is added, and vice versa, a version with a similar style but correct cause analysis is added), which is equivalent to doubling the number of our fine-tuning samples.

(c) Construct the chain-of-thought (CoT) data. Our fine-tuning data have both correct failure cause analysis and incorrect ones. When a human expert is faced with these two scenarios, he or she will first determine if that cause analysis is correct based on logic and find out the root cause of the test case failure based on the information provided, and then conceptualize a repair solution for that root cause and all the other causes, such as adding a new step, deleting a step, modifying an attribute in the locator, etc., and ultimately give the repaired and correct test case. In order to allow LLM to better learn the essence of human expert repair during fine-tuning, we purposely design the corresponding chain-of-thought reasoning for each set of fine-tuning data, which includes the thinking process as well as the suggested repair solutions mentioned above, and then we combine the chain-of-thought reasoning with the correctly repaired test cases to form the final answer.

When performing fine-tuning, we still use the default fine-tuning parameter settings and let the OpenAI platform automatically configure them for us. To be clear, the fine-tuning in this work is not based on the fine-tuned model in the *Major Elements Extraction* module.

2) *Intent Replication Analysis*

When there is no correct candidate among the generated repair candidates, LetTe will use the *Intent Replication Analysis* module to determine if there is still a possibility that the broken test cases can be fixed. Due to the ineffectiveness of multiple repair candidates, We consider whether the test case can still be correctly repaired. When a test case fails, the human expert typically refers to the locator in the web page to find out if the corresponding element is still there, or if a similar element exists, based on the information reported in the error. Therefore, LetTe extracts the filename of the test case and extracts the broken statement based on the error message. And together with the HTML source code of the old version, it analyzes the intent of the broken statement itself. Based on the filename, the intent of the broken statement, and the new version of the HTML source code, LetTe analyzes and determines if there is still a possibility that the test case can be correctly repaired. If the actions with the same intent cannot be reproduced, it output the analysis results. If the analysis concludes that the actions can be reproduced, we perform the repair of the test case by reapplying our approach.

Self-consistency checking: In order to get a trustworthy answer, inspired by Ahmed et al.'s study [14], we add self-consistency checking to this module by asking the same question five times. We select the one with the most consistent answer from all the generated binary results as the final answer. In other words, the minority obeys the majority.

C. *Implementation*

We use the most popular API of GPT-3.5-Turbo (gpt-3.5-turbo-0125), which is the base model for the fine-tuning model in our approach. We set the temperature parameter to 0.8 in all modules in Phase I and in the *Test Case Repair* module in Phase II, the same as in previous research work [15].

III. Evaluation

To evaluate our approach, we conduct experiments on five real-world Web applications. Furthermore, we apply the LetTe on a dataset used in the former research [16] and compare with the existing methods. Our four research questions (RQs) are shown below:

- RQ1: Is our approach effective for web test repairing?
- RQ2: How well does our approach compare to existing methods?
- RQ3: How efficient is our approach?
- RQ4: Can the test intent serve as a high-level guidance in web test repair?

For RQ1, we use two variants with few-shot learning based on our LetTe approach, and experimentally compare and analyze the impact of few-shot learning on web test repair as well as the contribution of the use of two fine-tuned LLMs in

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:51:56 UTC from IEEE Xplore. Restrictions apply.

our approach. For RQ2, we further demonstrate the effectiveness of our approach by applying it, with light modifications, to the web applications test cases used in previous research. For RQ3, we present the performance of our approach on generating repaired test cases, i.e., how many attempts are needed on average to repair the test case in each Web application. For RQ4, We conduct the ablation experiment to address whether the test intent actually serves as a high-level guidance to help the test repair process.

A. Experimental Setup

Dataset. In this paper, we adopt all of the open-source web applications as well as the test cases from the existing dataset [5] for RQ2, and we configure the corresponding runtime environments for the specific versions of these web applications. The detailed web application information is shown in Table 4. We execute the test cases to identify the broken test cases in the new version. Subsequently, we filter out the test cases that did not fail and those that do not require fixing like the study [5].

TABLE 4. APPLICATIONS FROM PREVIOUS RESEARCH DATASET

Application	ΔV	Original Version	Modified Version
AddressBook	30	4.0	6.1
Claroline	7	1.10.7	1.11.5
Collabtive	4	0.65	1.0
MantisBT	1	1.1.8	1.2.0
MRBS	12	1.2.6.1	1.4.9

Given that most of the versions selected for this existing dataset [5] were updated 10 years ago, we newly select some of the web applications and update the versions of the rest of the applications. For RQ1, RQ3 and RQ4, we collect five open-source web applications on Source Forge and GitHub and ensure that they have maintained and updated records in the last two years. Although the latest update of *Collabtive* was in 2017, at least two versions of it still exist to work properly, so we added it to our dataset as well. We use Python to write test cases for these web applications, and since existing research suggests [3] that only 1.88% of test case failures are related to assertions, we request to skip the writing of test assertions when writing the test cases, and we create a total of 336 test cases for the five web applications, and we manually count the number of functionalities in each web application and require the test cases to cover more than 60% of the functionalities of each web application. After testing, we obtain a total of 107 broken test cases. The statistics of our dataset are shown in Table 5.

TABLE 5. STATISTICS OF OPEN-SOURCE WEB APPLICATIONS IN OUR DATASET

Application	ΔV	Original Version	Modified Version	Test Cases	Broken Tests
Admidio	12	4.2.0	4.2.13	63	5
Collabtive	2	3.0.0	3.1.0	43	5
MantisBT	11	2.24.3	2.26.0	40	23
MRBS	2	1.11.1	1.11.4	30	5
PHP-Fusion	5	9.03.90	9.10.30	160	69

Metrics. We consider a test repair as passed if the repaired test case is able to run without errors and interruptions, and we consider a repair as correct if the repaired test case meets the test intent of the original test case while it is also passed. For RQ1 and RQ4, we use pass rate and correct repair rate as our evaluation metrics. And in RQ2, only the correct repair rate is used as the evaluation metric. Note that repairing test cases in RQ2 also includes repairing test assertions, which means that the test case repair with both test statements and assertions are correct is to be considered as a correct repair. The higher the repair rate score, the more effective the method is. For RQ3, we use Round-N as the evaluation metrics for this work, which represents the number of rounds needed to correctly repair the test case. The higher number of Round-N indicates better performance. The main reason why we do not choose the runtime as the evaluation criteria is that the VPN we use has significant network latency which may also leads to the timeout error.

Baseline. In RQ1 and RQ3, we modify our approach by constructing two variants to serve as a baseline for comparison with our approach. The difference between our approach and these two variants are showed in Table 6.

TABLE 6. THE DIFFERENCES BETWEEN LETTE VARIANTS

Methods	Fine-Tuning	Few-shot Learning
LetTe_{ntfs}	-	√
LetTe_{fs}	√	√
LetTe	√	-

In Section II, we apply fine-tuning and a series of prompting techniques in both the *Major Element Extraction* module and the *Test Case Repair* module. Therefore, we construct a variant that applies few-shot learning instead of fine-tuning in the *Major Element Extraction* module, while relying solely on zero-shot learning in the *Test Case Repair* module for getting the repair results. We name it LetTe_{ntfs}. To further explore the effectiveness of fine-tuning and few-shot learning on this task, we add the prompting method of few-shot learning on the basis of fine-tuning in the *Major Element Extraction* module, and the *Test Case Repair* module is consistent with our approach. This variant is named as LetTe_{fs}. In RQ2, we take the results of Lin et al. [5] as reference and select all the former methods from their comparison experiments, as a baseline to compare with our approach, i.e., WATER [4], VISTA [6], WEBEVO [8], SFTM [9] and UITESTFIX [5]. As for RQ4, we modify the input part of the *Test Case Repair* module in our approach, i.e., altering the prompt component for the LLM. Specifically, we change the original prompt "test intent: {content}" to "test intent: none". For the convenience of subsequent description, it will be referred to as LetTe_{ni}.

The dataset we manually collected and experimental results are available at https://github.com/Yingjie47/LetTe.

B. RQ1: Accuracy of Test Case Repair

To answer RQ1, we evaluate the accuracy of our approach on test case repair for web applications. For each web application in Table 5, we fix the broken test cases. After the test case repair results are generated, we manually check each

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:51:56 UTC from IEEE Xplore. Restrictions apply.

test case to see if it passes and whether it is a correct and reasonable repair.

Table 7 and Table 8 show the pass rate and correct repair rate of our approach on the test case repair. The accuracy of our approach is significantly better than the other variants in all metrics, i.e., when compared to the best variant (LetTe$_{fs}$), the pass rate and the correct repair rate of the test cases can be improved by 7.47% and 8.41%, respectively. Our approach LetTe is able to achieve 94.39% pass rate and 85.05% correct repair rate by relying on the fine-tuning and a series of prompting techniques.

TABLE 7. COMPARISON OF PASS RATES

Method	Web Applications					Average
	Admidio	Mantisbt	PHP-Fusion	Collabtive	MRBS	
LetTe$_{ntfs}$	100%	86.96%	75.36%	100%	80%	80.37%
LetTe$_{fs}$	100%	82.61%	86.96%	80%	100%	86.92%
LetTe	**100%**	**91.30%**	**94.20%**	**100%**	**100%**	**94.39%**

TABLE 8. COMPARISON OF CORRECT REPAIR RATES

Method	Web Applications					Average
	Admidio	Mantisbt	PHP-Fusion	Collabtive	MRBS	
LetTe$_{ntfs}$	100%	73.91%	60.87%	100%	80%	68.22%
LetTe$_{fs}$	100%	78.26%	72.46%	80%	100%	76.64%
LetTe	**100%**	**82.61%**	**82.61%**	**100%**	**100%**	**85.05%**

TABLE 9. COMPARISON OF REPAIR RATES WITH DIFFERENT BASELINES

Application	No. of Test Cases	Methods					
		WATER	VISTA	WEBEVO	SFTM	UITESTFIX	LetTe(Ours)
AddressBook	8	1	2	0	2	2	**6**
Claroline	29	0	24	0	0	24	**26**
Collabtive	5	**3**	1	2	0	**3**	2
MantisBT	38	18	4	19	6	24	**30**
MRBS	25	0	4	0	2	4	**15**
Total	105	22(21%)	35(33%)	21(20%)	10(10%)	57(54%)	**79(75%)**

Furthermore, we analyze the logging files generated during the experimental process and summarize the following three findings: (1) Comparing the two variants of our LetTe approach, it can be seen that the fine-tuning in our approach is effective; While both variants apply the few-shot learning prompting method in the *Major Element Extraction* module, our approach that applys the fine-tuning can lead to an increase of 6.55% in the pass rate and 8.42% in the correct repair rate, respectively. (2) Fine-tuning is more effective than few-shot learning in web test case repair task; Because of LLM's maximum token limitation, only one single sample can be provided as a demonstration at the time of prompting, which is not sufficient for web applications with numerous styles of interfaces as well as for the web test failures with 14 different kinds of reasons [3]. (3) Adding the few-shot learning prompts on the basis of already fine-tuned model is rather counterproductive to the accuracy of the method; Fine-tuning has already allowed LLM to learn how to extract the major elements, and applying few-shot learning to the prompt templates will only work well for a specific part of the broken test case repair process due to the singularity of the demonstration. At the same time, this will also limit the fine-tuned model's original reasoning ability to repair when encountering test cases outside of the few-shot learning examples.

C. RQ2: Effectiveness of LetTe Comparing with the existing methods

To answer RQ2, we conduct experiments on *AddressBook*, *Claroline*, *Collabtive*, *MantisBT* and *MRBS*, which have a total of 105 test cases to be fixed. The repair results of the experiment are shown in Table 9. The comparison of the repair rates reveals that our approach outperforms all the baselines, i.e., with an average repair rate of 75%.

Besides, in order to obtain a deeper understanding of why our approach outperforms other baselines, we summarize the following findings by investigating the log files generated during the repair process for some unique broken statements and assertions: (1) LetTe has a more concentrated search space in finding candidate repairs; WATER [4] tries to find any newly added form elements by analyzing the differences between the old and new versions during the repair process to provide suggestions for the repair, WEBEVO [8] makes use of CV techniques in addition to identifying changes in Web elements

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:51:56 UTC from IEEE Xplore. Restrictions apply.

at the document object model (DOM) level of the page, and UITESTFIX [5] makes use of the relationships between different elements and previous matching results to narrow down the search space. Our approach, on the other hand, identifies the base search space by comparing the differences, and then utilizes LLM's strengths in natural language processing to search for matches at the semantic and intent levels to narrow down the search space even further, as shown in some log files, where the valid repair candidates provided by LLM in one round of fixing even contain a large amount of similarities or duplicates. (2) LetTe generates repair candidates with more diverse locator expressions;} as we all know, Selenium can use multiple types of locators to find a page element. Also, each type of locator can have different expressions. LetTe takes advantage of this in the repair process. Based on the pre-training knowledge that the LLM already has, our approach provides more than one locator expression to locate the same element. This is done to increase the probability of a correct repair, especially for elements that cannot be located or cannot be interacted with using a single expression.

As to why our approach does not perform well enough on *Collabtive* test cases, we conclude the possible reasons as followed: (1) Test intent is not made for guiding match finding for a single specific element in our approach; UITESTFIX [5] utilizes the relative position of elements and their relationship to neighboring elements to improve the accuracy of matching. WATER [4] extracts five attributes for locating elements from the old DOM node and matches them in the new version. And LetTe, on the other hand, relies on intent to guide the repair logic in a macro scale. (2) Guided only by textual information such as test intent, it is not enough to fix test breakage that is caused by workflow changes; Our approach was initially designed to take only text-type data such as DOM information for test intent and failure cause analysis, which can sometimes be very complex and troublesome. This prevents our approach from fully utilizing its advantages, even with information such as test intent as a high-level guide. We hope to mitigate this problem in future work by employing multimodal large language models for visually enhanced analysis.

D. RQ3: Efficiency of the LetTe

To answer RQ3, we count the number of rounds required for LetTe to correctly fix the five web application test cases, and the results are shown in Figure 5. Note that this work counts the number of rounds required for a correct repair, so we remove the test cases that do not break and those that are only passed but not correctly repaired. Also, some of the test case repair processes includes multiple times of repairs to complete a full repair, so we take the highest number of rounds in these repair processes in our counting (e.g., broken statement 1 requires three rounds to repair, broken statement 2 requires one round to repair, we consider that this test case requires three rounds to be correctly repaired).

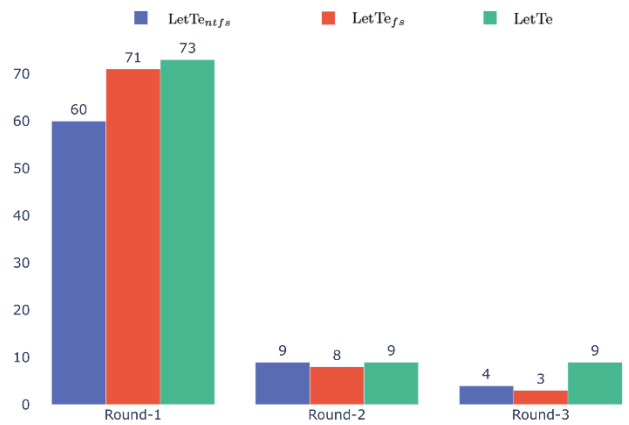

Figure. 5. Efficiency Comparison for Correct Repair.

Figure 5 demonstrates the efficiency of LetTe. Our approach has the highest number of test cases repaired in Round-1, up to 73, 2.8% higher than the best variant. By further analyzing the results in Figure 5, we summarize the following two findings: (1) The points made in RQ1 are proved once again, i.e., fine-tuning is effective in our approach and it is more effective than few-shot learning, and applying the few-shot learning prompting method on the basis of already applying fine-tuning will have negative effects. (2) Most of the broken test cases can be correctly repaired after the first application of our approach; 80.22% of the broken test cases were correctly repaired by LetTe after the first round of application of our approach, which we consider such efficiency is acceptable.

E. RQ4: Ablation Experiment for Test Intent

In order to demonstrate the role that test intent plays on web test repair in our approach, we conduct the ablation experiment. The results are shown in Table 10 and Table 11. It can be seen that after removing the test intent, the average value of pass rate decreases by no more than 2%, but the average value of correct repair rate decreases by about 12.15%. So the former is the result of sufficient a prior knowledge of Web testing, where LLM can do a good job of modifying broken test cases into test code that passes the run. And the latter is due to the lack of test intent as a high-level guidance. By examining the logging files, we find that there are quite a number of test cases where the major element extraction has accurately extracted the correct elements and their Xpaths from the new version during the repair process, but failed to effectively be utilized during the test case repair, resulting in LLM generating invalid repair candidates and thus the repair fails. So our finding is: The test intent is a good high-level guidance for web test repair, which helps to effectively utilize page information and guide the logical thinking through the repair process.

TABLE 10. COMPARISON OF PASS RATES

Method	Web Applications					Average
	Admidio	Mantisbt	PHP-Fusion	Collabtive	MRBS	
LetTe	100%	91.30%	94.20%	100%	100%	94.39%
LetTe$_{ni}$	100%	86.96%	92.75%	100%	100%	92.52%

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:51:56 UTC from IEEE Xplore. Restrictions apply.

TABLE 11. COMPARISON OF CORRECT REPAIR RATES

Method	Web Applications					Average
	Admidio	Mantisbt	PHP-Fusion	Collabtive	MRBS	
LetTe	100%	82.61%	82.61%	100%	100%	85.05%
LetTe$_{ni}$	80%	69.57%	71.01%	80%	100%	72.90%

IV. CONCLUSIONS

This paper proposed an intent-driven approach for automatic repair of web application test cases, named LetTe, using LLM with prompt engineering and fine-tuning. During the repair process, test intent is analyzed via LLM, and based on the chain-of-thought reasoning of manual repair logic, a fine-tuning model is constructed to fix broken test cases guided by the test intent. Specifically, we extract the major elements, analyze the test intent and the causes of test failure to guide the repairing of broken test cases, in which we design the corresponding fine-tuning data and prompt templates. To evaluate the effectiveness of LetTe, we conducted experiments on our collection of five real-world Web applications with 107 broken test cases, achieving a pass rate of 94.39% and a correct repair rate of 85.05%. LetTe also achieves a 75% correct repair rate on the dataset from the previous study. The performances of our approach are better than the baseline approaches.

In the future, we will continue to improve our LetTe to get better results. For example, nowadays GPT4 and Claude3 support the analysis of multimodal information, and information such as screenshots can also be used to assist in test case fixing to enhance the understanding of the causes of test failures.

REFERENCES

[1] Brian Robinson, Michael D. Ernst, Jeff H. Perkins, Vinay Augustine, and Nuo Li. Scaling up automated test generation: Automatically generating maintainable regression unit tests for programs. In 2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011), pages 23–32, 2011.

[2] Leandro Sales Pinto, Saurabh Sinha, and Alessandro Orso. Understanding myths and realities of test-suite evolution. In Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering, FSE '12, New York, NY, USA, 2012.

[3] Mouna Hammoudi, Gregg Rothermel, and Paolo Tonella. Why do record/replay tests of web applications break? 2016 IEEE International Conference on Software Testing, Verification and Validation (ICST), pages 180–190, 2016.

[4] Shauvik Roy Choudhary, Dan Zhao, Husayn Versee, and Alessandro Orso. Water: Web application test repair. In ETSE '11, 2011.

[5] Yuanzhang Lin, Guoyao Wen, and Xiang Gao. Automated fixing of web ui tests via iterative element matching. In 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pages 1188–1199, 2023.

[6] Andrea Stocco, Rahulkrishna Yandrapally, and Ali Mesbah. Visual web test repair. In Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2018, pages 503–514, New York, NY, USA, 2018.

[7] Mouna Hammoudi, Gregg Rothermel, and Andrea Stocco. Waterfall: an incremental approach for repairing record-replay tests of web applications. In Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, pages 751–762, New York, NY, USA, 2016. Association for Computing Machinery.

[8] Fei Shao, Rui Xu, Wasif Haque, Jingwei Xu, Ying Zhang, Wei Yang, Yanfang Ye, and Xusheng Xiao. Webevo: taming web application evolution via detecting semantic structure changes. In Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2021, pages 16–28, New York, NY, USA, 2021.

[9] Sacha Brisset, Romain Rouvoy, Lionel Seinturier, and Renaud Pawlak. Sftm: Fast matching of web pages using similarity-based flexible tree matching. Inf. Syst., 112(C), February 2023.

[10] Fangwen Mu, Xiao Chen, Lin Shi, Song Wang, and Qing Wang. Developer-intent driven code comment generation. In 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pages 768–780, 2023.

[11] Xiangyu Li, Marcelo d'Amorim, and Alessandro Orso. Intent-preserving test repair. In 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST), pages 217–227, 2019.

[12] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: deliberate problem solving with large language models. In Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23, Red Hook, NY, USA, 2024.

[13] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. ArXiv, abs/2002.08155, 2020.

[14] Toufique Ahmed and Premkumar Devanbu. Better patching using llm prompting, via self-consistency. In 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pages 1742–1746, 2023.

[15] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. Automated repair of programs from large language models. In Proceedings of the 45th International Conference on Software Engineering, ICSE '23, pages 1469–1481. IEEE Press, 2023.

[16] Javaria Imtiaz, Muhammad Zohaib Iqbal, and Muhammad Uzair khan. An automated model-based approach to repair test suites of evolving web applications. Journal of Systems and Software, 171:110841, 2021.

Authorized licensed use limited to: Trial User - National Taiwan University. Downloaded on March 13,2026 at 09:51:56 UTC from IEEE Xplore. Restrictions apply.